Multidimensional Temperature-Accelerated String Method (TASM)

Requirements: Users are expected to have a basic working knowledge of the following tools:

- GROMACS for running molecular dynamics simulations
- PLUMED for enhanced sampling simulations
- Bash scripting for automation and job execution
- Python for basic data processing and analysis

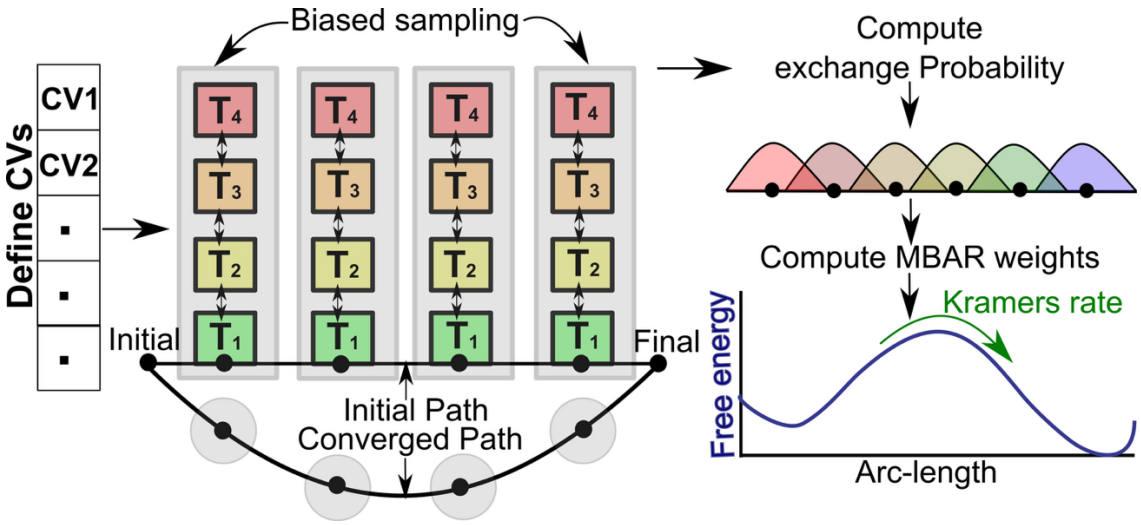


Figure 1: Schematic representation of free energy and kinetic estimation using the temperature-accelerated string method (TASM). A set of putative collective variables (CVs) is first defined, together with fixed initial and final metastable endpoints. An initial path connecting these endpoints is then generated using P discrete images. Each image is biased using a harmonic bias potential in multidimensional CV space and simulated at multiple temperatures, with replica exchange attempts performed at fixed intervals. The string is iteratively updated using the mean positions of the CVs until convergence to the minimum free-energy path (MFEP), following a strategy similar to that described previously (https://doi.org/10.1039/d2sm00888b). The retrospective exchange probability between sampled multidimensional phase-space regions is used as a proxy for phase-space overlap. After assurance of the non-zero overlap, MBAR weights are computed and used to reconstruct the free-energy profile along the arc length of the MFEP. Finally, the Kramers rate is estimated from the barrier height, local curvatures, and diffusivity.

We will follow the steps involved in Figure 1 sequentially.

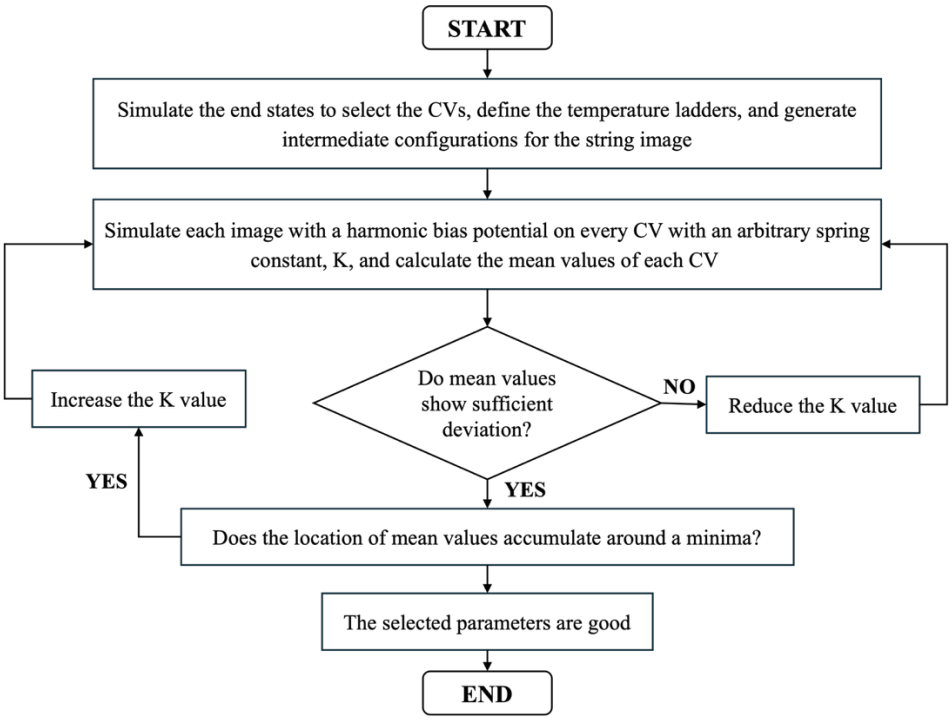
1. **Selecting the collective variables:** The first step is to identify the collective variables (CVs) that are relevant to the transition of interest. These CVs should describe the essential slow degrees of freedom associated with the transition pathway. To identify suitable CVs, unbiased molecular dynamics simulations should be performed for the known end states. The resulting trajectories can then be analyzed to determine correlations, relaxation times, and slow modes of motion. All slow variables that are weakly correlated with one another and directly relevant to the transition should be included in the CV definition for further analysis. A careful choice of CVs is essential, as the accuracy and efficiency of enhanced

sampling simulations depend strongly on how well the selected CVs capture the transition mechanism.

2. **Defining the parameters for TASM simulations:** The next step is to define the main parameters required for Temperature Accelerated String Method (TASM) simulations.
    a. **Number of temperature ladders:** Temperature biasing is applied to each image along the string to enhance the sampling of hidden slow degrees of freedom that are not explicitly included in the string CV coordinates. To choose the temperature ladder, short trial simulations should be performed at different temperatures. The goal is to determine an optimal temperature spacing between neighboring windows that gives an exchange probability of approximately 20%. The base temperature should be set to the target simulation temperature.
    b. **The number of string images:** The number of string images controls the resolution of the transition path and later determines the overlap between neighboring umbrella-sampling windows. There is no universal rule for selecting this number. A practical approach is to start with an initial number of images and then increase it if the phase-space overlap between neighboring windows is negligible or close to zero.
    c. **The force constant for the umbrella potential:** The force constant of the umbrella potential controls both the fluctuation of the string and the convergence speed of the string. To optimize this parameter, short simulations should be performed at each point of the initial string guess. The deviation of the mean sampled location from the initial position should then be analyzed. There is an important trade-off:
        i. A very high force constant restricts the system too strongly and may prevent sufficient deviation from the initial position, even when the image is far from the minimum free-energy path (MFEP).
        ii. A very low force constant leads to excessive fluctuations, causing the string images to drift toward one of the minima rather than remaining on the transition path.

    Therefore, the force constant should be chosen such that it allows sufficient deviation during biased sampling while still allowing the string to converge toward the MFEP.

Follow the flow chart below to generate the initial string and to decide the parameters for further string evolution



3. **Initiating the TASM string evolution:** After defining all required parameters and preparing the necessary intermediate configurations, the initial path guess can be evolved toward the minimum free-energy path (MFEP). The string evolution is performed using the provided Bash scripts.
    a. **First.sh:** This script takes the input configurations for each of the P string images, runs biased simulations for each window, and prepares the directories: md1, md2, ..., mdP. where P is the total number of string images.
    b. **Second.sh:** This script computes the mean value of each collective variable from every simulation directory and generates the updated CV values. The Python script new.py is used in this step.
    c. **Third.sh:** This script starts the first temperature-accelerated iteration. Each string image is simulated at multiple temperatures, and the COLVAR file from the base-temperature simulation is used to compute the mean CV values and evolve the string. The Python script equispacing.py is used to smooth the string and redistribute the string images with equal spacing.
    d. **Fourth.sh:** This script performs the same procedure as Third.sh, but automates the subsequent TASM iterations.
4. **Convergence Analysis:** The convergence of the string is evaluated by monitoring the absolute change in the string position between successive iterations. Use the Python script convergence.py to compute and analyze the string convergence. This script compares the string coordinates from different iterations and estimates how much the string has changed during the evolution. The string can be considered converged when the absolute change in the string position becomes sufficiently small over consecutive iterations.

5. **Exchange probability:** The retrospective exchange probability between neighboring windows is computed using the Python script exchange_prob.py. This analysis helps evaluate whether neighboring temperature or umbrella windows have sufficient overlap for reliable sampling. The input file format required by exchange_prob.py is the same as that used by the MBAR analysis code (https://github.com/avijeet0901/MBAR). The exchange probability should be checked between all neighboring windows. A reasonable exchange probability indicates sufficient overlap, while very low exchange probabilities suggest that additional intermediate windows may be required.
6. **MBAR implementation:** The final free-energy analysis is performed using the Multistate Bennett Acceptance Ratio (MBAR) method. Users are advised to follow the instructions provided in the MBAR repository to compute the statistical weights and estimate the free-energy surface (MBAR Repository: https://github.com/avijeet0901/MBAR). Ensure that the input files are prepared in the format required by the MBAR workflow before running the analysis.
7. **Free energy analysis:** The projected two-dimensional free energy surface is computed using the Python script 'error_2d_fes.py'. This script takes all COLVAR files containing the CV trajectories, together with the corresponding statistical weights obtained from the MBAR analysis. It also performs bootstrap analysis on the input data to estimate the uncertainty in the computed free energy surface. The one-dimensional free energy profile is then computed using the Python script '1d_error.py'. This script takes the two-dimensional free energy surface as input and projects it along the normalized arc length of the converged transition path, corresponding to the MFEP. The final output is a one-dimensional free energy profile along the transition pathway, including the estimated free energy error.
8. **Diffusivity computation:** To compute diffusivity along the transition path, first select representative transition-state configurations and run multiple short simulations initiated from these configurations. The Python script compute_RC.py is then used to compute the effective reaction coordinate, defined as the change in the normalized arc length along the converged path. This reaction coordinate quantifies the progress of the system along the transition pathway. Next, the Python script compute_msd.py is used to compute the average mean square displacement (MSD) as a function of time. Finally, the diffusivity is obtained using the Python script compute_D.py, which performs a linear fit to the MSD versus time data and estimates the diffusion coefficient along the reaction coordinate.
9. **Kramers rate estimation:** The effective Kramers rate is computed using the Python script kramers_rate.py. The required inputs are:
    a. The one-dimensional free energy profile,
    b. The diffusivity along the reaction coordinate
    c. An initial guess for the minimum location,
    d. An initial guess for the barrier location.

    Using these inputs, the script estimates the transition rate based on Kramers theory.